PNP Formal Reconstruction Report

Compiled Lean theorem inventory and current claim boundary

Coordinate `PNP-FORMAL-RECONSTRUCTION-STATUS-2026-07-20-61`

The repository does not currently establish $P = NP$.

Public theorem emission is disabled. A direct finite raw-machine verifier now proves concrete CNF-SAT membership in NP with an explicit polynomial bound. It does not prove CNF-SAT in P, NP-hardness, NP-completeness, or $P = NP$. The general charged pipeline is linked to the raw machine kernel, and the Cook–Levin formula has an exact externally sized answer-independent slot schedule plus direct fuelled bit and token specification cursors. A literal finite builder emits `FormulaWidth` copies of `T` followed by `F`, `Sep`, and the complete positive clause on variables zero, one, and two. It then executes the first padding transition and the entire remaining first-clause padding run, then emits the complete second clause `Sep F F F T F Finish`, traverses the entire remaining clause-two padding block, and then emits the complete third clause `Sep F F F T T F Finish` on variables zero and two, retaining its first in-range padding coordinate. Every traversed padding slot and emitted populated slot has a direct schedule proof, the emitted bits are the exact canonical prefix, and an external polynomial bounds the compiled run. The remaining clause-three padding, a general dynamic formula cursor, arbitrary raw slot decoder, and the remaining formula body are not implemented, so there is no complete raw polynomial-time formula builder or concrete CNFSAT reduction. The reviewed activation fingerprints are unset, no root theorem exists, and the abstract string-handle bridge is never an activation source.

Compiled declarations	9117
Theorem-kind declarations	4764
Assumption-free theorem-kind declarations	3122
Project-specific axioms	4
Concrete publication gate passed	**false**

Inventory SHA-256: `155a862474126aa8fb8c5c75c6e2e7126f1b69febac1e4100d505e405d974db5`

Generated from the pinned compiled Lean environment, not from source-text parsing.

Contents

1 Current authority and claim boundary

This report is the current repository report surface. Its factual theorem inventory comes from the compiled `PNP` environment under `leanprover/lean4:v4.31.0`. The exporter enumerates `Environment.constants`, resolves each originating module, and calls `Lean.collectAxioms` for every exported project declaration. The committed inventory is `status/LEAN_THEOREM_INVENTORY.json`; the public mirror is byte-identical.

The target remains $P = NP$, but target wording—including the new inactive definition `PNP.Main.ConcretePEqualsNP`—is not theorem evidence. JavaScript acceptance, Boolean fields, JSON strings, historical release records, report prose, and external review cannot activate a theorem. Only the separately derived concrete publication gate may control theorem emission, and that gate is currently false.

The completed direct CNF verifier consumes the literal canonical pair of an encoded formula and assignment certificate. Lean proves universal acceptance equivalence, rejection equivalence, and absence of timeout at its explicit polynomial fuel bound, and constructs a proof-bearing `PolynomialTimeVerifier CNFSAT`. Thus `PNP.Concrete.CNFSAT` is in the concrete NP class. This is a verifier theorem, not a deterministic polynomial-time SAT algorithm.

The concrete Cook–Levin development proves that the generated CNF formula is semantically equivalent to ordinary raw verifier execution in both input modes. It now also bounds the actual canonical unary-indexed formula encoding by an explicit fixed-verifier polynomial evaluated only at external source-input length. That output-size theorem does not execute the formula builder as a raw finite machine, prove its construction runtime, or package a concrete polynomial reduction.

The new rectangular schedule allocates exact constraint, clause, token, and raw-bit opportunities without reading the already-materialized program, formula, token encoding, or encoded formula as schedule inputs. Filtering populated slots reproduces those canonical objects, and the raw-bit slot count is exactly the external encoded-size polynomial. This remains a pure specification: no theorem charges constant time per slot, implements a raw builder, or proves a construction-runtime bound.

The direct formula cursor decodes constraint, clause, token, and bit coordinates without first constructing those complete canonical lists. Its nested options distinguish out-of-range lookup from valid empty padding, and Lean proves exact prefix, full, one-step-short, terminal, and excess-fuel behavior. Filtering the exact full cursor output yields the canonical encoding. These are specification-level evaluation theorems, not a constant-time raw slot interpreter or raw builder.

The first literal builder stage is a fixed 19-rule work machine. Starting at the proved all-input framer endpoint, it restores every source bit and appends exactly one unary tally symbol per bit in fresh right-side workspace. Its exact work cost is $2n^2 + 4n + 2$, and six-for-one compilation gives the exact raw cost $12n^2 + 24n + 12$. Malformed scan symbols and one-step-short fuel time out. This is only length preparation: it emits no formula bits and does not compose or refine a complete builder.

The executable input prefix now places the total framer and that tally in disjoint state images, with one total nine-symbol launch table between them. From every ordinary raw bitstring it reaches the same preserved-input unary-tally endpoint in exactly the sum of the two proved local traces plus one launch, and compilation is bounded by $18n^2 + 63n + 93$ in external input length. First-match isolation, timeout for the unused `zeroOne` tally scan symbol, and one-step-short timeout are explicit. No formula bit, cursor coordinate, or construction-time refinement is produced.

The standalone token appender carries one of the four canonical CNF tokens in its finite control state, scans the represented source, exact tally, and existing token suffix, writes exactly the selected two-bit symbol, and rewinds to the source focus. Its generic theorem quantifies every input, canonical prior token list, request, and exterior-left region. The distinguished start requests `T`; Lean proves direct formula-bit slots zero and one are populated true bits and that `CNFToken.t.bits = problem.encodedFormula.take 2` for every concrete tableau problem. The compiled first-token bound is $24n + 48$. Malformed tally/output symbols and one-step-short fuel time out.

The composed first-token prefix renames the complete 116-rule input prefix and complete 59-rule appender into disjoint injective state images and places nine symbol-preserving rules first in the literal table. Only the appender's accept/reject images are global halts. Every raw bitstring follows the exact framing, tally, launch, and append trace to the preserved workspace containing `[CNFToken.t]`. Its compiled external bound is $18n^2 + 87n + 147$, and blank-equivalent raw tapes reach the same encoded endpoint. Ending at the prefix endpoint before launch, malformed prefix or appender phases, and one-step-short fuel all remain timeout. This emits only two fixed formula bits: it does not compute the remaining header, interpret a dynamic cursor, or emit the complete formula.

The complete-header composition continues from that endpoint with a structurally generated unary `NatPolynomial` evaluator, a 16-rule unary-root controller, two complete 59-rule appender copies, and five total nine-symbol bridges under pairwise-disjoint injective state renamings. Its literal table has exactly `363 + evaluator.ruleCount` rules. Every raw input emits exactly `FormulaWidth` copies of `T` followed by `F`; the emitted bits equal `encodedFormula.take (2 * (FormulaWidth + 1))`, and an external `NatPolynomial` bounds the compiled run. This is the complete answer-independent width header only: dynamic cursor and formula-body emission, a complete builder refinement, and a concrete polynomial reduction remain absent.

The body-start composition continues from the complete header with a unary evaluator for the retained next-token coordinate, two total nine-symbol bridges, and one complete separator appender. Its literal table has exactly 440 plus the two evaluator rule counts. Every raw input reaches `T` repeated `FormulaWidth` times, followed by `F` and `Sep`; its bits equal the canonical encoded-formula prefix through that separator. The token coordinate is two past the formula variable slot bound and the corresponding bit coordinate is twice it. This retained coordinate is data, not an executable dynamic cursor; all subsequent body emission remains absent.

The first-literal composition continues from that endpoint with a third unary evaluator, three total nine-symbol bridges, and complete fixed `T` and `F` appender copies. Its literal table has exactly 585 plus the width, body-start next-slot, and first-literal next-slot evaluator rule counts. Every raw input reaches `T` repeated `FormulaWidth` times, followed by `F`, `Sep`, `T`, and `F`. A constructive schedule proof pins the final pair to the positive sign and unary-zero terminator of the first at-least-one shape-clause literal: positive variable zero. The emitted bits equal `encodedFormula.take (2 * (FormulaWidth + 4))`. The retained next token coordinate is four past the formula variable slot bound, but it remains data rather than an executable dynamic cursor; the rest of the formula body remains absent.

The first-clause composition continues from that endpoint with a fourth unary evaluator and a fixed 535-rule tail appender assembled from eight complete token-appender copies and seven total bridges. The global literal table has exactly 1138 plus the width, body-start next-slot, first-literal next-slot, and first-clause next-slot evaluator rule counts. Every raw input reaches `T` repeated `FormulaWidth` times, followed by `F`, `Sep`, `T`, `F`, `T`, `T`, `F`, `T`, `T`, `T`, `F`, and `Finish`. A constructive schedule proof identifies this as the complete positive at-least-one shape clause on variables zero, one, and two. The emitted bits equal `encodedFormula.take (2 * (FormulaWidth + 12))`; the retained next token coordinate is twelve past the formula variable slot bound. It remains data rather than an executable dynamic cursor, and the remaining formula body is absent.

One literal token-cursor step continues from that first-clause endpoint. A total nine-symbol launch enters a fixed 45-rule cursor-advance table, giving a global table of exactly 1192 plus the four inherited unary-evaluator rule counts. Every raw input preserves the emitted clause while moving the retained unary coordinate from `FormulaVariableSlotBound + 12` to `FormulaVariableSlotBound + 13`. The direct decoder and token-level specification cursor prove that the consumed coordinate is the first in-range padding opportunity and therefore emits no token. Including launch, the suffix costs exactly `2*cursorWord.length + 8` work steps; the compiled run is bounded by `FirstClausePrefix.rawTimeBound + 48 + 12*cursorWord.length`. Malformed scratch, the unlaunched predecessor endpoint, and one-step-short total fuel remain timeout. This is one fixed padding transition, not a general cursor loop or arbitrary schedule decoder.

The remaining-padding composition continues from that one-step endpoint with two structurally generated unary evaluators and one fixed 25-rule countdown controller. Writing `D = (FormulaVariableSlotBound - 1) * (FormulaVariableSlotBound + 6)`, its literal table has exactly 1244 plus the six inherited and generated evaluator rule counts. Every raw input traverses all `D` remaining first-clause padding coordinates without emitting a token and reaches `FormulaVariableSlotBound + 1 + FormulaTokensPerClause`. Direct lookup at that coordinate is proved to return `Sep`, and the recursive specification run agrees across every no-emission step and the following separator observation. The exact compiled run has a verifier-fixed external input-size polynomial bound; malformed countdown scratch/root states, the unlaunched endpoint, and one-step-short fuel remain timeout. This identifies the second-clause boundary but does not emit the second clause or implement a general formula cursor.

The second-clause-separator composition continues with a selected 59-rule `Sep` appender, two total nine-symbol bridges, and the existing fixed 45-rule cursor advance. Its literal table has exactly 1366 plus the six inherited and generated evaluator rule counts. Every raw input emits the canonical prefix through the separator beginning clause two and advances the retained unary coordinate by one; direct lookup proves the following token is `F`. The emitted bits equal `encodedFormula.take (2 * (FormulaWidth + 13))`. The compiled run is bounded by the predecessor bound plus `246 + 24*n + 12*FormulaWidth + 12*cursorWord.length`. The combined 56-declaration audit has 15 empty closures, 11 using only `propext`, and 30 using only `propext` and `Quot.sound`. Malformed appender tally/output, malformed cursor scratch, both unlaunched endpoints, and one-step-short total fuel remain timeout. This is one fixed populated transition, not a general dynamic cursor or remaining-body builder, and it does not emit the following `F`.

Two further fixed compositions emit the complete negative literals on variables zero and one in clause two. They use selected `F`, `T`, and `F` appender copies, fixed cursor advances, and total symbol-preserving bridges. Exact schedule proofs identify each sign, unary index, and literal terminator; the second composition retains the following `Finish`.

The complete-second-clause composition emits that retained `Finish` with one selected 59-rule appender and advances the cursor once. Its suffix has exactly 113 rules, while the global table has exactly 2098 plus the six inherited and generated evaluator rule counts. Every raw input emits bits equal to `encodedFormula.take (2 * (FormulaWidth + 19))`. Direct lookup proves that the executed coordinate is the clause terminator and the retained next coordinate is in-range padding. The new external raw bound is the predecessor bound plus `390 + 24*n + 12*FormulaWidth + 12*cursorWord.length`. The exact 57-line audit has 15 empty, 10 `propext`, and 32 `propext/Quot.sound` closures. This completes clause two but does not by itself traverse its padding or implement a general formula cursor.

The second-clause-padding composition evaluates `D = (V - 1) * (V + 6) + 5`. With `C` abbreviating `formulaTokensPerClause`, Lean proves `D = C - 7`, traverses exactly those remaining padding opportunities with the reused 25-rule countdown, and evaluates the target coordinate. Three total bridges and two fixed unary evaluators yield a global table with exactly 2150 plus the six inherited/generated evaluator counts and the two new evaluator counts. Every raw input reaches `V + 1 + 2 * formulaTokensPerClause`; every traversed coordinate is direct padding and the retained coordinate is the third clause's opening `Sep`. No token is emitted, so the bits remain `encodedFormula.take (2 * (FormulaWidth + 19))`. The exact 68-line audit has 26 empty, 9 `propext`, and 33 `propext/Quot.sound` closures. This reaches clause three only as a retained coordinate; it does not emit the separator or implement a general formula cursor.

The third-clause-separator composition reuses the selected 59-rule `Sep` appender and fixed 45-rule cursor advance behind one new total bridge. Its global table has exactly 2272 plus the eight inherited evaluator rule counts. Every raw input emits the separator beginning clause three, advances the retained coordinate to `V + 1 + 2 * formulaTokensPerClause + 1`, and proves the following direct token is `F`. The emitted bits equal `encodedFormula.take (2 * (FormulaWidth + 20))`. Its external bound adds `330 + 24*n + 12*FormulaWidth + 12*cursorWord.length` to the predecessor bound. The exact 56-line audit has 14 empty, 11 `propext`, and 31 `propext/Quot.sound` closures. This emits one fixed separator but does not emit the following `F`, complete clause three, or implement a general formula cursor.

The third-clause-first-literal composition reuses the audited 235-rule two-`F` appender/cursor suffix behind one total bridge. Its global table has exactly 2516 plus the eight inherited evaluator rule counts. Every raw input emits the complete negative literal on variable zero in clause three and retains `V + 1 + 2 * formulaTokensPerClause + 3`, the negative-sign coordinate on variable two. Constructive schedule proofs identify the third excluded pair as zero and two and prove that the emitted sign, unary-zero terminator, and retained sign are all `F`. The emitted bits equal `encodedFormula.take (2 * (FormulaWidth + 22))`. Its external bound adds `732 + 48*n + 24*FormulaWidth + 24*cursorWord.length` to the separator bound. The exact 87-line audit has 24 empty, 18 `propext`, and 45 `propext/Quot.sound` closures. This emits one fixed literal but does not emit the next negative sign, complete clause three, or implement a general formula cursor.

The third-clause-second-literal composition adds a fixed 479-rule `F T T F` appender/cursor suffix behind one total bridge. Its nested component tables have 113, 235, 357, and 479 rules, and its global table has exactly 3004 plus the eight inherited evaluator rule counts. Every raw input emits the complete negative literal on variable two and retains `V + 1 + 2 * formulaTokensPerClause + 7`, the following clause-terminator coordinate. Direct and specification schedule proofs establish `F`, `T`, `T`, `F`, and the following `Finish`. The emitted bits equal `encodedFormula.take (2 * (FormulaWidth + 26))`. Its external bound adds `1752 + 96*n + 48*FormulaWidth + 48*cursorWord.length` to the first-literal bound. The exact 145-line audit has 46 empty, 32 `propext`, and 67 `propext/Quot.sound` closures. This emits the second fixed literal but does not emit the following `Finish`, complete clause three, or implement a general formula cursor.

Machine field	Current value
`mathematicalTheoremEstablished`	false
`publicTheoremEmissionAllowed`	false
`finalTheoremReady`	false
`publicTheoremStatement`	`null`
`rootLeanTheorem`	`PNP.Main.p_eq_np`
`rootLeanTheoremPresent`	false

2 Concrete publication gate

The compatibility entry is `PNP.Main.p_eq_np`; a matching name alone is insufficient. The separate publication target is `PNP.Main.ConcretePEqualsNP`. Eligibility also requires the exact theorem type, reviewed kernel fingerprints, and a tightly fixed Lean-standard axiom closure. The target is present as an inactive definition backed by finite charged-pipeline semantics; its presence is not a proof, and the compiler/refinement link to the raw machine kernel remains an explicit blocker for the general charged-pipeline target. The direct CNF verifier is one closed raw-machine instance; it does not supply that general link.

The expected fingerprints are intentionally unset. This is a fail-closed migration gate, not a missing runtime input. A verifier must reject any attempt to set `passed=true` while a fingerprint is null, while the concrete model is ineligible, or while any other subcheck is false.

Gate subcheck	Result
standardComplexityModelEligible	true
concreteTargetPresent	true
concreteTargetIsDefinition	true

Gate subcheck	Result
concreteTargetKernelTypeFingerprintConfigured	false
concreteTargetKernelTypeFingerprintMatches	false
concreteTargetKernelValueFingerprintConfigured	false
concreteTargetKernelValueFingerprintMatches	false
compatibilityRootPresent	false
compatibilityRootIsTheorem	false
compatibilityRootHasExactConcreteType	false
compatibilityRootKernelTypeFingerprintConfigured	false
compatibilityRootKernelTypeFingerprintMatches	false
axiomClosureFingerprintConfigured	false
axiomClosureFingerprintMatches	false
sourceClosureFingerprintConfigured	false
sourceClosureFingerprintMatches	false
axiomClosureUsesOnlyLeanStandardAllowlist	false

Allowed foundation axioms are fixed to `Classical.choice`, `Quot.sound`, and `propext`. Project axioms, `sorryAx`, or any unknown axiom make the gate fail. The four currently disclosed project axioms are:

- `PNP.CheckPCCPackexp`
- `PNP.GeneratePCCPack`
- `PNP.LockedNANDThreshold`
- `PNP.ResidualBandExactMinimization`

2.1 Why the abstract bridge is ineligible

The current `PNP.PEqualsNP` abbreviation compares witness classes whose language and algorithm structures carry string names or code handles rather than concrete execution semantics and proved polynomial bounds. Consequently, even a kernel-clean theorem of that abstract type would not establish the standard P-versus-NP statement. It is compatibility information only.

The separate `PNP.Concrete.PEqualsNP` target uses proof-bearing P and NP witnesses, canonical input/certificate pairing, finite program syntax, and charged polynomial runtime and output bounds. This is a substantive formalization milestone, but it is not yet proved equivalent to execution in the raw machine kernel. SAT completeness, SAT in P, and the root theorem also remain absent.

3 Formal milestone ledger

Each earned row requires exact compiled names and theorem kinds, empty axiom closures, per-name domain-separated kernel-type SHA-256 matches for 1089 detailed candidates, and the pinned whole-Lean source closure. Human-readable scope remains conservative: type or source drift revokes credit.

Milestone	Status	Exact scope	Boundary
Concrete machine and cost kernel	formalized-foundation-only	One literal finite four-stage pipeline handles every raw bitstring. The total framer uses exactly 4 work steps on empty input, 4 * k * k + 9 * k + 7 for complete two-bit cells, and 4 * k * k + 9 * k + 5 when the final cell is partial; its compiled raw cost is bounded by 6 * m * m + 39 * m + 75. Every supplied exact n-step target run lifts to exactly 3 * n work steps. PipelineCompiler preserves one raw target's verdict and ordinary output at an explicit external polynomial. PipelineSequentialCompiler composes two raw targets in one literal finite table, passes either first verdict onward, preserves the second verdict and output, retains stuck-first timeout, and proves R(m) = PipelineRaw(p)(m) + 6 + PipelineRaw(q)(m + p(m) + 1). PipelineRefinement recursively applies that compiler to every FunctionProgram composition and DecisionProgram precomposition node. Its 16 audited declarations have empty axiom closure, and PolynomialTimeDecider.compileToMachine exposes the complete tree as one raw polynomial-time machine.	This closes the concrete complexity machine-link blocker only. It does not construct a deterministic polynomial-time CNF-SAT decider or an NP-completeness reduction. CNF-SAT in P, NP-completeness, and P = NP are not established; PNP.Main.p_eq_np remains absent and the publication gate remains false.
Concrete P, NP, and polynomial reductions	formalized	Bitstring languages, finite charged decision/verifier/function pipelines grounded in concrete machines, polynomial certificate/runtime/output bounds, recursively compiled exact raw-machine refinements, many-one reductions, and the NP-complete-in-P implication.	The recursive refinement compiler closes the charged-to-raw machine link, but it does not construct raw paired verifiers beyond the existing CNF-SAT membership witness, prove CNF-SAT is in P or NP-complete, or establish the publication root theorem.

Milestone	Status	Exact scope	Boundary
Concrete universal CNF-SAT verifier	formalized-np-membership-only	An unconditional formula-grammar outcome, universal bounded work outcome, exact accept/reject semantics, no-timeout theorem, literal finite-machine paired verifier, and CNF-SAT membership in NP.	This proves CNF-SAT membership in NP only; it does not prove CNF-SAT is in P, NP-completeness, or P = NP.
Concrete Cook-Levin dimensions and variable layout	formalized-foundation-only	Fuel padding preserves designated halts and stuck timeouts; every literal machine state is below an explicit finite ceiling; the input, time, and tape dimensions are executable NatPolynomial expressions; and tape-symbol, head, state, certificate-bit, and certificate-length variables occupy proved disjoint in-range blocks. Elementary at-least-one and pair-exclusion CNF clauses have exact propositional semantics. All 20 reviewed theorems have empty axiom closure.	This is the finite layout substrate only. It does not yet construct a transition tableau, prove tableau soundness or completeness, compile a formula emitter, define a polynomial reduction to CNFSAT, or establish CNFSAT NP-completeness, CNFSAT in P, or P = NP.
Concrete fixed-certificate execution tableau	formalized-foundation-only	For one fixed source input, certificate, finite-rule machine, and fuel budget, the canonical raw execution trace has exactly fuel + 1 configurations. Intrinsic first-match transition validity is equivalent to equality with that trace, every valid endpoint equals the ordinary run endpoint, and accept/reject/timeout verdicts agree exactly with boundedDecide. All 30 declarations and eight reviewed theorem types have empty axiom closure.	This is the semantic fixed-certificate tableau only. It does not yet quantify a variable certificate, encode tape windows or tableau rows as CNF, compile a formula emitter, define a polynomial reduction to CNFSAT, or establish CNFSAT NP-completeness, CNFSAT in P, or P = NP.

Milestone	Status	Exact scope	Boundary
Uniform bounded verifier tableaux	formalized-foundation-only	For an arbitrary proof-bearing verifier in the concrete NP model and one source input, the complete finite decision pipeline is compiled to one raw machine. Every certificate inside the explicit certificate polynomial shares one answer-independent raw fuel obtained by evaluating the compiled raw-time polynomial at the maximum encoded input size. Per-certificate fuel is below that bound, padding follows a proved halt, exact verdicts are preserved, and language membership is equivalent to an existential bounded accepting semantic tableau. All 41 declarations and nine reviewed theorem types have empty axiom closure.	This is a uniform semantic verifier-tableau equivalence only. It does not encode configurations, transition windows, or variable-length certificates as Boolean clauses; emit CNF; prove emitter size or runtime polynomials; define a polynomial reduction to CNFSAT; or establish CNFSAT NP-completeness, CNFSAT in P, or P = NP.
Finite local Cook-Levin CNF compiler	formalized-foundation-only	Width-indexed literals materialize exact Boolean assignments and compile to the concrete CNF semantics without out-of-range variables. Unit constraints, finite implications, and pairwise exactly-one groups have exact satisfaction equivalences; local programs have exact recursive clause counts, satisfiability reflection, and a proved well-scoped formula. All 75 declarations and nine reviewed theorem types have empty axiom closure.	This is a local clause compiler only. It does not enumerate the verifier tableau, encode initialization, transition windows, certificate length, or acceptance as a complete formula; prove an all-input emitter size/runtime polynomial; define a reduction to CNFSAT; or establish CNFSAT NP-completeness, CNFSAT in P, or P = NP.

Milestone	Status	Exact scope	Boundary
Finite whole-tableau Cook-Levin CNF syntax	formalized-foundation-only	A uniform bounded verifier-tableau problem is compiled answer-independently into one finite, well-scoped concrete CNF formula. The program emits one-hot symbol/head/state rows, first-match control updates, untouched-cell preservation, exact input-only or symbolic paired-certificate initialization, and a designated final accept constraint. Canonical encoding/decoding, exact local satisfaction reflection, and exact emitted clause counting are proved. All 81 declarations and ten reviewed theorem types have empty axiom closure.	This is finite formula syntax and local-clause reflection only. It does not yet prove that formula satisfiability is equivalent to an accepting raw verifier execution, prove external encoded-output-size or construction-runtime polynomials, define a concrete polynomial reduction to CNFSAT, or establish CNFSAT NP-completeness, CNFSAT in P, or P = NP.
Finite whole-tableau Cook-Levin CNF semantics	formalized-foundation-only	Every satisfying assignment decodes to a unique-symbol, unique-head, unique-state finite row at each bounded time. The decoded initial row is exact in both verifier modes, each adjacent row is the literal deterministic first-match successor, and the final state is accepting. Conversely, every such intrinsic finite accepting tableau constructs a satisfying assignment. Formula satisfiability and concrete CNFSAT membership are therefore equivalent to this finite semantics. All 161 explicit declarations are axiom-audited; the six reviewed theorem types depend only on the permitted Lean standard axioms Quot.sound and propext, with no project axiom.	This milestone itself proves equivalence only with an intrinsic finite-row model; the following raw-tape milestone supplies the execution bridge. External encoded-output-size and construction-runtime polynomials, a concrete polynomial reduction to CNFSAT, CNFSAT NP-completeness, CNFSAT in P, and P = NP remain absent.

Milestone	Status	Exact scope	Boundary
Finite tableau to raw Tape semantics	formalized-foundation-only	The finite Cook-Levin rows are proved exactly equivalent to ordinary focused raw Tape execution. Absolute two-sided tape observations cover explicit and implicit blanks; the local successor matches designated halts, missing-rule stutter, and the literal first matching raw rule; a centered head invariant rules out finite-window clamping; and both verifier input modes have exact initial rows, including reversible bounded-certificate reconstruction. Formula satisfiability and concrete CNFSAT membership are therefore equivalent to the concrete verifier language. All 54 explicit declarations are axiom-audited, and the nine reviewed theorem types use only the permitted Lean standard axiom allowlist with no project axiom.	This proves exact semantic reduction correctness only. It does not yet prove external encoded-formula-size or formula-construction-runtime polynomials, package a concrete PolynomialReduction, establish CNFSAT NP-completeness, CNFSAT in P, or P = NP.
External Cook-Levin encoded-formula size	formalized-foundation-only	The actual canonical unary-indexed CNF bitstring generated for a fixed concrete polynomial-time verifier is bounded by an explicit NatPolynomial evaluated only at the external source-input length. Exact codec lengths, both verifier input modes, every concrete tableau constraint family, program and emitted-clause counts, clause width, and the mode-sensitive exact formula-variable width polynomial are covered. All 110 explicit declarations are axiom-audited; the ten reviewed theorem types use only the permitted Lean standard axioms Quot.sound and propext, with no project axiom or choice axiom.	This does not implement or time a raw finite formula builder, package a concrete PolynomialReduction, establish CNFSAT NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Rectangular Cook-Levin formula schedule	formalized-foundation-only	For every concrete verifier-tableau problem, a finite answer-independent rectangular schedule allocates exact constraint, clause, token, and raw-bit slots. Removing empty slots in order reproduces the existing canonical local program, bounded clauses, CNF tokens, and encoded formula. The raw-bit schedule length is exactly the previously proved encodedFormulaSizePolynomial evaluated at external source-input length. All 79 explicit declarations are axiom-audited; the eight reviewed theorem types use only the permitted Lean standard axioms Quot.sound and propext, with no project axiom or choice axiom.	This is a pure schedule specification. It does not interpret a slot as a constant-time raw-machine action, implement or time a raw finite formula builder, construct a FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Direct Cook-Levin formula cursor	formalized-foundation-only	For every concrete verifier-tableau problem and natural coordinate, direct constraint, clause, token, and raw-bit decoders agree pointwise with the canonical rectangular schedules while preserving out-of-range, valid-padding, and populated states. Fuelled bit traversal proves exact bounded prefixes, complete traversal, one-step-short behavior, terminal behavior, excess-fuel stability, the external encoded-size polynomial, and canonical emitted output. A token-level specification cursor additionally exposes exact in-range, done, and terminal single-step laws. All 136 explicit declarations are axiom-audited; the 16 reviewed theorem types use only the permitted Lean standard axioms Quot.sound and propext, with no project axiom or choice axiom.	This is a Lean specification cursor. It does not prove constant-time raw slot interpretation, implement or time a raw finite formula builder, construct a FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Literal Cook-Levin builder input-length tally	formalized-foundation-only	A fixed 19-rule work machine starts at the proved all-input framer endpoint, preserves every external source bit, appends exactly one unary tally symbol per source bit in fresh right-side workspace, and returns to the source head. It accepts after exactly 2*n*n + 4*n + 2 work steps; the compiled raw machine reaches the encoded endpoint after exactly 12*n*n + 24*n + 12 steps. Malformed internal scan symbols and one-step-short fuel both time out. All 39 public declarations are axiom-audited; the ten reviewed theorem types use only the permitted Lean standard axioms Quot.sound and propext, with no project or choice axiom.	This is only the input-length preparation stage. It does not emit formula bits, interpret direct cursor slots as raw transitions, compose a complete raw formula builder, construct a FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-completeness, establish CNFSAT in P, or prove P = NP.
Executable Cook-Levin builder input prefix	formalized-foundation-only	One literal finite work machine contains collision-free renamed copies of the total all-input framer and fixed 19-rule unary tally, with a total nine-symbol tape/head-preserving launch between them. Every raw bitstring reaches the exact preserved-input tally endpoint in totalInputFramerWorkSteps(input) + 1 + 2*n*n + 4*n + 2 work transitions and within the external compiled polynomial 18*n*n + 63*n + 93. All 40 public declarations are axiom-audited: 29 have empty closure, one uses only propext, and ten use only propext and Quot.sound.	This is still only an executable input-preparation prefix. It emits no formula bit, performs no raw formula-cursor interpretation, supplies no complete formula builder or construction-time RawRefinement, packages no concrete PolynomialReduction, and establishes neither CNFSAT NP-completeness, CNFSAT in P, nor P = NP.

Milestone	Status	Exact scope	Boundary
Standalone Cook-Levin builder token appender	formalized-foundation-only	A fixed 59-rule work machine appends any of the four canonical two-bit CNF tokens selected only by its finite control state after a represented source word and exact unary input-length tally. For source length n and k existing tokens, it accepts at the exact endpoint after 2 * (max 1 n + n + k + 3) work steps. Its distinguished start state appends T, the first canonical formula header token, within the external compiled bound 24*n + 48; the two emitted bits are proved equal to the first two bits of every concrete verifier-tableau encoding. Malformed tally/output phases and one-step-short fuel remain timeouts. All 68 public declarations are axiom-audited: 42 have empty closure, 13 use only propext, and 13 use only propext and Quot.sound.	This milestone audits the token appender independently of its later composed use. It does not compute the remaining width header, interpret a dynamic formula cursor coordinate, emit a complete formula, construct a builder RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Composed Cook-Levin builder first-token prefix	formalized-foundation-only	One literal 184-rule finite work machine contains all 116 executable input-prefix rules, nine total symbol-preserving bridge rules, and all 59 token-appender rules under injective disjoint state renamings. Every raw bitstring runs through total framing, exact unary input tallying, the bridge, and the appender to preserve the workspace and emit exactly the first T header token. The exact all-input work trace compiles within 18*n*n + 87*n + 147 raw steps; the final two bits equal the first two bits of every canonical encoded verifier-tableau formula. Prefix-endpoint, malformed tally/output, and one-step-short negative cases time out. All 37 public declarations are axiom-audited: 21 have empty closure, three use only propext, and 13 use only propext and Quot.sound, with no project axiom or Classical.choice.	This emits exactly the fixed answer-independent first T token, hence only the first two canonical formula bits. It does not compute the remaining width header, implement a dynamic cursor, emit a complete formula, construct a builder RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Composed Cook-Levin complete width header	formalized-foundation-only	For every fixed concrete polynomial-time verifier, one literal finite work machine contains the 184-rule raw-input/first-token prefix, a structurally generated unary NatPolynomial evaluator, a 16-rule unary-root controller, two complete 59-rule token appenders, and five total nine-symbol bridges under injective pairwise-disjoint state renamings. Its table has exactly 363 plus the evaluator rule count rules. Every raw input follows an exact all-input trace, evaluates the verifier's mode-sensitive formula width into unary scratch space without calling NatPolynomial.eval in executable rules, and emits exactly FormulaWidth copies of T followed by F. The resulting bits are the complete canonical encoded-formula width header, and an external NatPolynomial bounds the compiled raw run. The evaluator's 74 and composition's 84 public declarations are completely axiom-audited: every closure is empty or uses only propext and Quot.sound, with no project axiom or Classical.choice.	This endpoint is the complete answer-independent width header only. It does not implement the dynamic formula cursor or body emission, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Composed Cook-Levin body-start prefix	formalized-foundation-only	For every fixed concrete polynomial-time verifier, one literal finite work machine contains the complete width-header machine, a structurally generated unary evaluator for the next canonical token slot, one complete 59-rule token appender, and two total nine-symbol bridges under injective pairwise-disjoint state renamings. Its table has exactly 440 plus the width-evaluator and next-token-slot-evaluator rule counts. Every raw input follows an exact all-input trace, preserves the unary input tally and header workspace, retains next token coordinate FormulaVariableSlotBound + 2 and bit cursor twice that coordinate, and emits exactly FormulaWidth copies of T followed by F and Sep. The resulting bits are the canonical encoded-formula prefix through the first body separator; the compiled run is bounded by the complete-header bound plus 72, six times the unary next-slot evaluator work count, 24*n, and 12*width. All 60 public declarations are axiom-audited: 30 have empty closure, five use only propext, and 25 use only propext and Quot.sound, with no project axiom or Classical.choice.	This milestone emits only the fixed answer-independent body-opening separator after the complete width header and retains its coordinate as data. It does not implement a dynamic formula cursor or subsequent body emission, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Composed Cook-Levin first-literal prefix	formalized-foundation-only	For every fixed concrete polynomial-time verifier, one literal finite work machine contains the complete body-start-prefix machine, a structurally generated unary evaluator for token coordinate FormulaVariableSlotBound + 4, two complete 59-rule token appenders, and three total nine-symbol bridges under four injective pairwise-disjoint state renamings. Its table has exactly 585 plus the width, body-start next-slot, and first-literal next-slot evaluator rule counts. Every raw input follows an exact all-input trace, preserves the unary tally and earlier token workspace, retains the corresponding doubled bit cursor, and emits exactly FormulaWidth copies of T followed by F, Sep, T, and F. The final T/F pair is constructively pinned to the positive sign and unary-zero terminator of the first scheduled literal, positive variable zero, and all emitted bits equal encodedFormula.take (2 * (FormulaWidth + 4)). The compiled run is bounded by the body-start bound plus 174, six times the new unary evaluator work count, 48*n, and 24*width. All 74 current public declarations are axiom-audited: 21 have empty closure, three use only propext, and 50 use only propext and Quot.sound, with no project axiom or Classical.choice; the added halt-separation interface supports the reviewed first-clause composition.	This milestone emits only the first canonical literal after the body-opening separator and retains the following coordinate as data. It does not implement a dynamic formula cursor or remaining body emission, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Composed Cook-Levin first-clause prefix	formalized-foundation-only	For every fixed concrete polynomial-time verifier, one literal finite work machine contains the complete first-literal-prefix machine, a structurally generated unary evaluator for token coordinate FormulaVariableSlotBound + 12, and a fixed 535-rule tail made from eight complete 59-rule token appenders and seven total nine-symbol bridges under disjoint injective state renamings. Its table has exactly 1138 plus the width, body-start next-slot, first-literal next-slot, and first-clause next-slot evaluator rule counts. Every raw input follows an exact all-input trace, preserves the unary tally and earlier token workspace, retains the corresponding doubled bit cursor, and emits exactly FormulaWidth copies of T followed by F, Sep, T, F, T, T, F, T, T, T, F, and Finish. The final eight tokens are constructively pinned to the remainder of the complete positive at-least-one shape clause on variables zero, one, and two; the following direct token slot is proved to be valid padding. All emitted bits equal encodedFormula.take (2 * (FormulaWidth + 12)). The compiled run is bounded by the first-literal bound plus 1158, six times the new unary evaluator work count, 192*n, and 96*width. All 79 public declarations are axiom-audited: 38 have empty closure, 13 use only propext, and 28 use only propext and Quot.sound, with no project axiom or Classical.choice; the 80-line combined audit additionally covers the predecessor halt-separation theorem.	This milestone emits only the complete first canonical clause after the width header and retains the following coordinate as data. It does not implement a dynamic formula cursor or remaining formula-body emission, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Literal Cook-Levin token-cursor padding step	formalized-foundation-only	For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete first-clause prefix with a total nine-symbol bridge and a fixed 45-rule cursor-advance table under disjoint nested state images. Its table has exactly 1192 plus the four inherited unary-evaluator rule counts. Every raw input follows an exact all-input trace, preserves the complete first-clause output, and advances the retained unary token coordinate from FormulaVariableSlotBound + 12 to FormulaVariableSlotBound + 13. The direct decoder proves that the consumed coordinate is the first in-range padding slot, so the token-level specification cursor advances without emitting a token. The cursor suffix takes exactly 2*cursorWord.length + 8 work steps including launch, and the compiled run is bounded by FirstClausePrefix.rawTimeBound + 48 + 12*cursorWord.length. All 47 public declarations, including two reviewed dead-state dispatch facts used downstream, are axiom-audited: 14 have empty closure, seven use only propext, and 26 use only propext and Quot.sound, with no project axiom or Classical.choice.	This is one fixed padding-coordinate transition after the complete first clause. It is not a general dynamic cursor loop or raw decoder for arbitrary schedule coordinates, emits no additional formula token, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Cook-Levin first-clause remaining-padding run	formalized-foundation-only	For every fixed concrete polynomial-time verifier, one literal finite work machine composes the preceding first-clause cursor step with structurally generated unary evaluators for D = (FormulaVariableSlotBound - 1) * (FormulaVariableSlotBound + 6) and the second-clause coordinate, plus one fixed 25-rule countdown controller. Its table has exactly 1244 plus the six inherited and generated evaluator rule counts. Every raw input follows an exact all-input trace, preserves the complete first-clause output, traverses all D remaining first-clause padding coordinates without emitting a token, and reaches FormulaVariableSlotBound + 1 + FormulaTokensPerClause, where direct schedule lookup is proved to return Sep. The recursive specification run agrees at every padding coordinate and the following specification step observes Sep. The compiled exact run has an explicit external input-size polynomial bound, accepts within that bound, and remains fail-closed for malformed countdown scratch/root states, the unlaunched prefix endpoint, and one-step-short fuel. All 83 public declarations plus one predecessor transport theorem are axiom-audited: 37 have empty closure, 11 use only propext, and 36 use only propext and Quot.sound, with no project axiom or Classical.choice.	This milestone executes exactly the remaining first-clause padding block and identifies the second-clause separator boundary. It is not a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the second clause or remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Cook-Levin second-clause separator step	formalized-foundation-only	For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete first-clause padding run with a selected 59-rule Sep appender, two total nine-symbol bridges, and the existing fixed 45-rule cursor advance under collision-free nested state images. Its table has exactly 1366 plus the six inherited and generated unary-evaluator rule counts. Every raw input follows an exact all-input trace, emits the canonical token prefix through the Sep beginning clause two, and advances the retained unary coordinate by one. The emitted bits equal encodedFormula.take (2 * (FormulaWidth + 13)), and direct schedule lookup proves that the following token is F. The compiled run is bounded by BuilderFirstClausePaddingRun.rawTimeBound + 246 + 24*n + 12*FormulaWidth + 12*cursorWord.length. Malformed appender tally/output, malformed cursor scratch, both unlaunched endpoints, and one-step-short total fuel remain timeout. All 54 new public declarations plus two reviewed predecessor dispatch facts are axiom-audited: 15 have empty closure, 11 use only propext, and 30 use only propext and Quot.sound, with no project axiom or Classical.choice.	This milestone emits only the fixed populated Sep transition beginning clause two and advances to the following F coordinate. It is not a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit that F or the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Cook-Levin second-clause first-literal prefix	formalized-foundation-only	For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete second-clause-separator prefix with two selected 59-rule F appenders, four total symbol-preserving bridges, and two copies of the existing 45-rule cursor advance under collision-free nested state images. One F/cursor component has 113 rules, the two-component suffix has 235 rules, and the global table has exactly 1610 plus the six inherited and generated unary-evaluator rule counts. Every raw input follows an exact all-input trace, emits the canonical prefix through the complete negative literal on variable zero in clause two, and retains the negative-sign coordinate on variable one. The emitted bits equal encodedFormula.take (2 * (FormulaWidth + 15)); direct schedule lookup proves the sign, unary-zero terminator, and following sign are all F. The compiled run is bounded by BuilderSecondClauseSeparatorStep.rawTimeBound + 564 + 48*n + 24*FormulaWidth + 24*cursorWord.length. All four pre-launch endpoints, malformed tally/output in both appenders, malformed scratch in both cursors, and one-step-short fuel remain timeout. All 85 new public declarations plus two reviewed cursor dead-loop facts are axiom-audited: 25 have empty closure, 18 use only propext, and 44 use only propext and Quot.sound, with no project axiom or Classical.choice.	This milestone emits only the fixed negative literal on variable zero in clause two and advances to the following negative-sign coordinate. It does not complete clause two, implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, emit the remaining formula body, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Cook-Levin second-clause second-literal prefix	formalized-foundation-only	For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete clause-two first-literal prefix with selected 59-rule F, T, and F appenders, six total symbol-preserving bridges, and three copies of the existing 45-rule cursor advance under collision-free nested state images. The selected T/cursor component has 113 rules, the T/F tail has 235 rules, the complete three-token suffix has 357 rules, and the global table has exactly 1976 plus the six inherited and generated unary-evaluator rule counts. Every raw input follows an exact all-input trace, emits the canonical prefix through the complete negative literal on variable one in clause two, and retains the following Finish coordinate. The emitted bits equal encodedFormula.take (2 * (FormulaWidth + 18)); direct schedule lookup proves the sign F, unary unit T, terminator F, and following Finish. The compiled run is bounded by BuilderSecondClause-FirstLiteralPrefix.rawTimeBound + 1026 + 72*n + 36*FormulaWidth + 36*cursorWord.length. All six pre-launch endpoints, malformed tally/output in all three appenders, malformed scratch in all three cursors, and one-step-short fuel remain timeout. All 113 new public declarations plus two reviewed cursor dead-loop facts are axiom-audited: 34 have empty closure, 25 use only propext, and 56 use only propext and Quot.sound, with no project axiom or Classical.choice.	This milestone emits only the fixed negative literal on variable one in clause two and advances to the following Finish coordinate. It does not emit the clause terminator, complete clause two, implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, emit the remaining formula body, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Cook-Levin complete second-clause prefix	formalized-foundation-only	For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete clause-two second-literal prefix with a selected 59-rule Finish appender, two total symbol-preserving bridges, and one copy of the existing 45-rule cursor advance under collision-free state images. The fixed suffix has exactly 113 rules and the global table has exactly 2098 plus the six inherited and generated unary-evaluator rule counts. Every raw input follows an exact all-input trace, emits the canonical prefix through the complete second clause, and retains its first in-range padding coordinate. The emitted bits equal encodedFormula.take (2 * (FormulaWidth + 19)); direct schedule lookup proves the executed opportunity is Finish and the retained next opportunity is padding. The compiled run is bounded by BuilderSecondClauseSecondLiteralPrefix.rawTimeBound + 390 + 24*n + 12*FormulaWidth + 12*cursorWord.length. Both pre-launch endpoints, malformed appender tally/output, malformed cursor scratch, and one-step-short fuel remain timeout. All 55 new public declarations plus two reviewed cursor dead-loop facts are axiom-audited: 15 have empty closure, 10 use only propext, and 32 use only propext and Quot.sound, with no project axiom or Classical.choice.	This milestone emits only the fixed Finish terminator that completes clause two and advances to its first padding coordinate. It does not traverse clause-two padding, reach clause three, implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, emit the remaining formula body, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Cook-Levin second-clause remaining-padding run	formalized-foundation-only	For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete second-clause prefix with structurally generated unary evaluators for D = (FormulaVariableSlotBound - 1) * (FormulaVariableSlotBound + 6) + 5 and the third-clause coordinate, plus the reused fixed 25-rule PaddingCountdown controller. Three total symbol-preserving WorkChain bridges yield a table with exactly 2150 plus the six inherited/generated predecessor evaluator rule counts and the two new evaluator rule counts. Every raw input follows an exact all-input trace, preserves the complete second-clause output, traverses all D = FormulaTokensPerClause - 7 remaining second-clause padding coordinates without emitting a token, and reaches FormulaVariableSlotBound + 1 + 2 * FormulaTokensPerClause, where direct schedule lookup is proved to return Sep. The recursive specification run agrees at every padding coordinate and the following specification step observes Sep. The emitted bits remain encodedFormula.take (2 * (FormulaWidth + 19)). The compiled exact run is bounded by BuilderSecondClausePrefix.rawTimeBound + 18 + 6 * countEvaluator.workSteps + 6 * (D * (2 * countRootPrefixLength + 8) + D * D) + 6 * targetEvaluator.workSteps, accepts within that external input-size polynomial, and remains fail-closed for malformed countdown scratch/root states, the unlaunched prefix endpoint, and	This milestone executes exactly the remaining second-clause padding block and identifies the third-clause separator boundary without emitting a token. It reaches clause three only as a retained coordinate. It is not a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit the third-clause separator or remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Cook-Levin third-clause separator step	formalized-foundation-only	For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete second-clause padding run with the reused selected 59-rule Sep appender, two total nine-symbol bridges, and the existing fixed 45-rule cursor advance under collision-free nested state images. Its table has exactly 2272 plus eight inherited unary-evaluator rule counts. Every raw input follows an exact all-input trace, emits the canonical token prefix through the Sep beginning clause three, and advances the retained unary coordinate by one. The emitted bits equal encodedFormula.take (2 * (FormulaWidth + 20)), and direct schedule lookup proves that the following token is F. The compiled run is bounded by BuilderSecondClausePaddingRun.rawTimeBound + 330 + 24*n + 12*FormulaWidth + 12*cursorWord.length. All 48 new public declarations plus eight reviewed suffix interfaces are axiom-audited: 14 have empty closure, 11 use only propext, and 31 use only propext and Quot.sound, with no project axiom or Classical.choice.	This milestone emits only the fixed populated Sep transition beginning clause three and advances to the following F coordinate. It is not a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, does not emit that F or the remaining formula body, and does not construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Cook-Levin third-clause first-literal prefix	formalized-foundation-only	For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete third-clause-separator prefix with the reused 235-rule two-F appender/cursor suffix behind one total nine-symbol bridge under collision-free nested state images. One F/cursor component has 113 rules, the two-component suffix has 235 rules, and the global table has exactly 2516 plus the eight inherited and generated unary-evaluator rule counts. Every raw input follows an exact all-input trace, emits the canonical prefix through the complete negative literal on variable zero in clause three, and retains the negative-sign coordinate on variable two. The emitted bits equal encodedFormula.take (2 * (FormulaWidth + 22)); direct schedule lookup proves the sign, unary-zero terminator, and following sign are all F, with the third excluded pair constructively identified as variables zero and two. The compiled run is bounded by BuilderThirdClauseSeparatorStep.rawTimeBound + 732 + 48*n + 24*FormulaWidth + 24*cursorWord.length. All four pre-launch endpoints, malformed tally/output in both appenders, malformed scratch in both cursors, and one-step-short fuel remain timeout. All 74 new public declarations plus eleven reviewed suffix declarations and two reviewed cursor dead-loop facts are axiom-audited: 24 have empty closure, 18 use only propext, and 45 use only propext and Quot.sound, with no project axiom or Classical.choice.	This milestone emits only the fixed negative literal on variable zero in clause three and advances to the following negative-sign coordinate. It does not emit that following F, complete clause three, implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, emit the remaining formula body, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Cook-Levin third-clause second-literal prefix	formalized-foundation-only	For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete third-clause-first-literal prefix with a fixed 479-rule F/T/T/F appender/cursor suffix behind one total nine-symbol bridge under collision-free nested state images. The nested component tables have 113, 235, 357, and 479 rules, and the global table has exactly 3004 plus the eight inherited and generated unary-evaluator rule counts. Every raw input follows an exact all-input trace, emits the canonical prefix through the complete negative literal on variable two in clause three, and retains the following Finish coordinate. The emitted bits equal encodedFormula.take (2 * (FormulaWidth + 26)); direct schedule lookup proves the sign F, both unary units T, the terminator F, and the following Finish. The compiled run is bounded by BuilderThirdClauseFirstLiteralPrefix.rawTimeBound + 1752 + 96*n + 48*FormulaWidth + 48*cursorWord.length. All eight pre-launch endpoints, malformed tally/output in all four appenders, malformed scratch in all four cursors, and one-step-short fuel remain timeout. All 145 public declarations are axiom-audited: 46 have empty closure, 32 use only propext, and 67 use only propext and Quot.sound, with no project axiom or Classical.choice.	This milestone emits only the fixed negative literal on variable two in clause three and advances to the following Finish coordinate. It does not emit the following Finish, complete clause three, implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, emit the remaining formula body, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Cook-Levin complete third-clause prefix	formalized-foundation-only	For every fixed concrete polynomial-time verifier, one literal finite work machine composes the complete clause-three second-literal prefix with a selected 59-rule Finish appender, two total symbol-preserving bridges, and one copy of the existing 45-rule cursor advance under collision-free state images. The fixed suffix has exactly 113 rules and the global table has exactly 3126 plus the eight inherited unary-evaluator rule counts. Every raw input follows an exact all-input trace, emits the canonical prefix through the complete third clause, and retains its first in-range padding coordinate. The emitted bits equal encodedFormula.take (2 * (FormulaWidth + 27)); direct schedule lookup proves the executed opportunity is Finish and the retained next opportunity is padding. The compiled run is bounded by BuilderThirdClauseSecondLiteralPrefix.rawTimeBound + 498 + 24*n + 12*FormulaWidth + 12*BuilderThirdClauseSeparatorStep.cursorWord.length. Both pre-launch endpoints, malformed appender tally/output, malformed cursor scratch, and one-step-short fuel remain timeout. All 55 new public declarations plus two reviewed cursor dead-loop facts are axiom-audited: 15 have empty closure, 10 use only propext, and 32 use only propext and Quot.sound, with no project axiom or Classical.choice.	This milestone emits only the fixed Finish terminator that completes clause three and advances to its first padding coordinate. It does not traverse clause-three padding, implement a general dynamic formula cursor or raw decoder for arbitrary schedule coordinates, emit the remaining formula body, construct a complete formula builder or FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-hardness or NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Typed direct-wire NAND semantics	formalized	Typed topological Boolean NAND programs and ordered multi-output direct-wire semantics.	This does not establish circuit minimization, SAT, or P = NP.
Finite enumeration, equivalence, and reference minimum	formalized	Exhaustive finite Boolean direct-wire search under the empty-profile reference model.	No polynomial-runtime result is obtained from this exhaustive reference search.
Concrete framed replacement and slack	formalized	The concrete serial framed context with support outputs and explicit bypass wires.	This is not an arbitrary-support replacement theorem for the report's global family.
Locked-NAND local candidates and baseline accounting	formalized	Typed local macro candidates, source-derived counts, and five finite local square baselines.	Local minima are not a global BaselineDistinct or locked-NAND threshold theorem.
Conditional locked-NAND threshold boundary	formalized-with-premises	A proof-bearing six-premise candidate boundary for an arbitrary satisfiable proposition.	The premises are not instantiated by a uniform SAT-to-locked-NAND builder.
Fail-closed explicit-list residual routes	formalized-explicit-list-only	Executable strict-gain search over a caller-supplied finite implementation list.	Unresolved excludes no gain outside the supplied list and cannot imply ZeroSlack.
Global locked-NAND construction and threshold	not-formalized	A uniform answer-independent SAT instance builder, global baseline, trace equivalence, and threshold.	The conditional boundary does not discharge this missing theorem.
Global ZeroSlack, PCCMin, and polynomial runtime	not-formalized	Complete residual routing, global ZeroSlack contradiction, exact minimization, and polynomial bounds.	Proof-bearing result structures and explicit-list failure do not supply global completeness.
Concrete standard P-versus-NP target and root theorem	not-formalized	Raw-machine-linked complexity classes, concrete SAT completeness and a SAT decider, plus the publication root theorem.	The concrete target definition remains inactive: CNF-SAT now has a raw-machine verifier and NP-membership proof, but a deterministic polynomial-time CNF-SAT decider, NP-completeness transport, and PNP.Main.p_eq_np remain absent; the legacy string-handle bridge is ineligible.

4 Compiled inventory summary

The inventory contains 9117 exported `PNP.*` kernel declarations from 81 modules. It excludes 3151 private compiler auxiliaries. Counts describe the compiled environment; they do not measure mathematical completeness.

Module	Declarations	Theorems
PNP.Bridge	93	22
PNP.Complexity	93	20
PNP.Concrete.BitString	123	60
PNP.Concrete.CNF	247	60
PNP.Concrete.CNFVerifier	10	7
PNP.Concrete.CNFWorkCorrectness	858	524
PNP.Concrete.CNFWorkFrameCorrectness	16	4
PNP.Concrete.CNFWorkInput	29	14
PNP.Concrete.CNFWorkMachine	85	3
PNP.Concrete.CNFWorkTransitions	64	63
PNP.Concrete.CNFWorkUniversalCorrectness	294	145
PNP.Concrete.Complexity	312	92
PNP.Concrete.CookLevinBuilderBodyStartPrefix	86	67
PNP.Concrete.CookLevinBuilderCompleteHeader	134	85
PNP.Concrete.CookLevinBuilderDynamicTokenCursorStep	68	54
PNP.Concrete.CookLevinBuilderFirstClausePaddingRun	160	114
PNP.Concrete.CookLevinBuilderFirstClausePrefix	121	86
PNP.Concrete.CookLevinBuilderFirstLiteralPrefix	125	103
PNP.Concrete.CookLevinBuilderFirstTokenPrefix	48	36
PNP.Concrete.CookLevinBuilderInputLength	49	27
PNP.Concrete.CookLevinBuilderInputPrefix	52	39
PNP.Concrete. CookLevinBuilderSecondClauseFirstLiteralPrefix	135	106
PNP.Concrete.CookLevinBuilderSecondClausePaddingRun	126	92
PNP.Concrete.CookLevinBuilderSecondClausePrefix	86	69
PNP.Concrete. CookLevinBuilderSecondClauseSecondLiteralPrefix	158	118
PNP.Concrete.CookLevinBuilderSecondClauseSeparatorStep	86	69
PNP.Concrete.CookLevinBuilderThirdClauseFirstLiteralPrefix	131	105
PNP.Concrete.CookLevinBuilderThirdClausePrefix	93	76
PNP.Concrete. CookLevinBuilderThirdClauseSecondLiteralPrefix	213	158
PNP.Concrete.CookLevinBuilderThirdClauseSeparatorStep	81	66
PNP.Concrete.CookLevinBuilderTokenAppender	105	70
PNP.Concrete.CookLevinBuilderUnaryPolynomial	216	127
PNP.Concrete.CookLevinFormulaCursor	259	121
PNP.Concrete.CookLevinFormulaSchedule	112	81
PNP.Concrete.CookLevinFormulaSize	168	149
PNP.Concrete.CookLevinLayout	204	71
PNP.Concrete.CookLevinLocalCNF	172	53
PNP.Concrete.CookLevinRawTapeBridge	109	97
PNP.Concrete.CookLevinTableau	71	23
PNP.Concrete.CookLevinTableauCNF	179	60
PNP.Concrete.CookLevinTableauCNFSemantics	190	136
PNP.Concrete.CookLevinVerifierTableau	58	25
PNP.Concrete.Machine	284	67
PNP.Concrete.PipelineCompiler	38	27
PNP.Concrete.PipelineInputFramer	106	18
PNP.Concrete.PipelineMachineSimulation	175	79
PNP.Concrete.PipelineOutputHandoff	23	4
PNP.Concrete.PipelinePairedCompiler	29	25
PNP.Concrete.PipelineRefinement	68	24
PNP.Concrete.PipelineSequentialCompiler	36	28
PNP.Concrete.PipelineSequentialStateNamespace	30	21
PNP.Concrete.PipelineStageBridges	77	59
PNP.Concrete.PipelineStateNamespace	77	34
PNP.Concrete.PipelineTapeGeometry	20	12
PNP.Concrete.PipelineTerminalBridge	73	62
PNP.Concrete.TapeBlankEquivalence	25	17
PNP.Concrete.TapeHandoff	18	11
PNP.Concrete.Target	2	1
PNP.Concrete.TerminalOutputPacker	111	36
PNP.Concrete.WorkInput	23	14
PNP.Concrete.WorkMachine	308	108
PNP.DirectWireBaseline	48	25
PNP.LockedNAND	11	4
PNP.LockedNANDBaseline	57	22
PNP.LockedNANDDirect	84	57
PNP.LockedNANDLocalBaseline	30	22

Module	Declarations	Theorems
PNP.LockedNANDMacros	288	98
PNP.LockedNANDPrefix	84	40
PNP.LockedNANDThresholdBoundary	60	35
PNP.Main	36	12
PNP.NANDComposition	77	38
PNP.NANDEnumerator	120	46
PNP.NANDMinimum	56	23
PNP.NANDSemantics	198	80
PNP.NANDSlack	15	12
PNP.NANDTruthTable	72	28
PNP.PCCMin	44	8
PNP.ResidualBand	9	2
PNP.ResidualRoutes	141	46
PNP.SAT	4	1
PNP.ZeroSlack	141	21

5 Remaining formal blockers

Six formal blockers remain active:

1. `Formal.ConcreteSAT`
2. `Formal.LockedNANDThreshold`
3. `Formal.ResidualBandMinimizer`
4. `Formal.ZeroSlack`
5. `Formal.PolynomialRuntimeAndCertificateBounds`
6. `Formal.RootTheoremAndAxiomAudit`

The conditional locked-NAND threshold module assumes typed baseline and full candidates, baseline conditions, preservation of existing outputs, an unsatisfiable final-zero law, and satisfiable final-output laws. The explicit residual scanner searches only a caller-supplied finite list. Neither result constructs the missing global reduction or proves global ZeroSlack, polynomial PCCMin, SAT in P, or P equals NP.

6 Historical report provenance

Earlier 56-page report revisions stated a direct checker-mediated P-versus-NP claim. Those bytes remain historical audit material at their pinned legacy coordinates and through the explicit archive and supersession records. They are not imported into, quoted as authority by, or used to generate this report. File identity and replay of historical predicates do not establish the named mathematical propositions.

Source tag	`final-pnp-proof-report-hardened-7072f8d`
Source commit	`7072f8d0bda6d44d240f9bb3fad624fd357e1278`
Archive manifest	`archive/legacy-v0/ARCHIVE.json`

7 Verification

The current reconstruction can be checked with:

```
lake build PNP
node scripts/export-lean-theorem-inventory.mjs --check
node scripts/generate-formal-publication.mjs --check
node pcc-formal-reconstruction-status0.mjs --json
npm run pnp:verify -- --no-write
npm run report:check
```

The inventory coordinate is `PNP-LEAN-THEOREM-INVENTORY-2026-07-20-61`. Its exact byte digest is `155a862474126aa8fb8c5c75c6e2e7126f1b69febac1e4100d505e405d974db5`. The report is regenerated from that inventory and the reviewed publication map. The reviewed milestone source-closure SHA-256 is `0d09467c09dbdd99b07c0fea2f21e24d75b9efc4701c6d5c6e3102a913cba0c8`; the computed and pinned values match. Successful regeneration confirms consistency of these status surfaces; it does not turn an absent concrete theorem into a proof.